Anatomía de un LLM

Fundamentos matemáticos de los modelos de lenguaje

Fase VI · Diseño y construcción de optimizadores con IA

Módulo 28

LLM como optimizadores y del prototipo a la producción

Búsqueda evolutiva de programas, optimización de sistemas con LLM, AutoML, MLOps, valor y riesgo, y guía de diseño

---

Resumen del módulo

Edición de octubre de 2026

Sobre este resumen

Este documento reúne lo indispensable de los apuntes del Módulo 28: el texto expositivo, todas las definiciones y todos los resultados (teoremas, proposiciones, lemas y corolarios), las tablas y las observaciones de contenido. Se han suprimido las demostraciones, los ejemplos, las lecturas, las figuras, los ejercicios y sus soluciones. La numeración de temas, secciones, enunciados y ecuaciones es la misma que en los apuntes, de modo que cualquier referencia puede buscarse allí; las que remiten a elementos suprimidos (un ejemplo, una figura, una ecuación de una demostración) conservan su número en los apuntes.

Notación

Símbolo	Significado
$\mathcal{P}$, $\pi \in \mathcal{P}$	espacio de programas (o de instrucciones, configuraciones) y un elemento
$h(\pi)$, $\hat{h}(\pi)$	puntuación verdadera del programa y puntuación medida por el evaluador
q	probabilidad de que una propuesta mejore al mejor programa actual
C, s_j, r, $g = r - s$	capacidad del contenedor, tamaño del objeto j, capacidad restante y hueco tras colocar
$L_1 = \lceil \sum_j s_j / C \rceil$	cota inferior del número óptimo de contenedores
K, n	número de candidatos comparados y tamaño del conjunto de validación
η, $r_{\min}$, $r_{\max}$	factor de reducción y presupuestos mínimo y máximo de la multifidelidad
$R(t)$, $F(t)$, $f(t)$, $\lambda(t)$	fiabilidad, función de distribución, densidad y tasa de fallo de una vida útil
β, θ	parámetros de forma y de escala de la distribución de Weibull
c_p, c_f	coste de una intervención preventiva y de una avería
PSI, KL	índice de estabilidad poblacional y divergencia de Kullback–Leibler
VAN, ρ	valor actual neto y tipo de descuento anual
CVaR_α	valor en riesgo condicional al nivel α
€, h, d, mm/s, ºC	euros, horas, días, milímetros por segundo (vibración) y grados Celsius

Índice

Sobre este resumen **ii**

1 Búsqueda evolutiva de programas guiada por LLM **1**
1.1 Optimizar en el espacio de programas . . . 1
1.2 FunSearch . . . 2
1.3 AlphaEvolve . . . 3
1.4 Por qué funciona y cuándo no . . . 3
1.5 Empaquetado en línea . . . 4
1.6 Caso de estudio: una heurística de empaquetado generada . . . 4

2 Optimización de instrucciones, de sistemas con LLM y de código **6**
2.1 Instrucciones como variables de decisión . . . 6
2.2 El sesgo del ganador . . . 7
2.3 Caso de estudio: clasificación de partes de avería . . . 7
2.4 La autoconsistencia como problema de optimización . . . 7
2.5 Optimización de código con agentes . . . 8

3 AutoML y búsqueda de arquitecturas **10**
3.1 AutoML como optimización sobre programas . . . 10
3.2 Búsqueda de arquitecturas neuronales . . . 11
3.3 Multifidelidad y coste . . . 11
3.4 Caso de estudio: AutoML del clasificador de averías . . . 12

4 Del prototipo a la producción: arquitectura, MLOps y evaluación **13**
4.1 Arquitectura de un optimizador en producción . . . 13
4.2 Deuda técnica en sistemas con aprendizaje automático . . . 14
4.3 Deriva y reentrenamiento . . . 14
4.4 Evaluar decisiones sin experimentar . . . 15
4.5 Fiabilidad y mantenimiento . . . 16
4.6 Caso de estudio: mantenimiento predictivo de una línea de envasado . . . 16

5 Cuantificar el valor y el riesgo **19**
5.1 Coste y beneficio . . . 19
5.2 Intervalos de confianza para la mejora . . . 19
5.3 Caso de estudio: ¿cuánto vale el optimizador de mantenimiento? . . . 20
5.4 Robustez y riesgo de cola . . . 20
5.5 Seguridad, supervisión y modos de fallo . . . 21
5.6 El Reglamento europeo de inteligencia artificial . . . 21

6 Guía para construir un optimizador con IA en cualquier ámbito **23**
6.1 Los doce pasos . . . 23
6.2 Plantilla de documento de diseño . . . 24
6.3 Mini-caso A: secuenciación de formatos en una fábrica de bebidas . . . 24
6.4 Mini-caso B: presupuesto de geometría en un videojuego . . . 24
6.5 Mini-caso C: asignación de trabajos en un centro de datos . . . 24

Bibliografía **26**

Tema 1

Búsqueda evolutiva de programas guiada por LLM

En muchos problemas lo que se busca no es un número ni un vector, sino un *procedimiento*: una regla de prioridad para un planificador, una heurística de empaquetado, un núcleo de cálculo más rápido. Este tema estudia cómo un modelo de lenguaje, encerrado en un bucle evolutivo con un evaluador automático, puede buscar en el espacio de programas. Sigue el artículo de FunSearch de Romera-Paredes *et al.* [44] y su implementación de referencia [16], el informe de AlphaEvolve de Novikov *et al.* [34] y la nota técnica de Google DeepMind [14]; para la representación de programas como expresiones se apoya en el capítulo «Expression Optimization» de Kochenderfer y Wheeler [23], y para el empaquetado, en la §3.3 de Williamson y Shmoys [50]. La lectura crítica de Herrmann y Pallez [17] sirve de contrapeso.

1.1 Optimizar en el espacio de programas

Definición 1.1 (Búsqueda de programas). Sea $\mathcal{P}$ el conjunto de programas que respetan una firma dada (por ejemplo, funciones que reciben un objeto y la lista de contenedores abiertos y devuelven una puntuación por contenedor). Sea $\mathcal{D}$ una distribución de instancias del problema y $u(\pi, x) \in \mathbb{R} \cup \{-\infty\}$ la utilidad de ejecutar π sobre la instancia x, con $u = -\infty$ si el programa falla, excede el tiempo o produce una salida inválida. La *búsqueda de programas* es

$$\max_{\pi \in \mathcal{P}} \; h(\pi), \qquad h(\pi) = \mathbb{E}_{x \sim \mathcal{D}}\big[u(\pi, x)\big].$$

Como $\mathcal{D}$ no se conoce, se usa un *evaluador* $\hat{h}(\pi) = \frac{1}{m}\sum_{i=1}^{m} u(\pi, x_i)$ sobre instancias de entrenamiento $x_1, \ldots, x_m$, y la calidad final se mide con instancias de prueba independientes.

El problema es de caja negra en el sentido de la definición 5.1 del módulo 24: no hay gradiente, el espacio es discreto y enorme, y cada evaluación cuesta una ejecución. La programación genética clásica busca en ese espacio con mutaciones y cruces aleatorios de árboles sintácticos [23]; su dificultad es que casi todas las mutaciones aleatorias producen programas inútiles. La idea de FunSearch es sustituir la mutación aleatoria por un modelo de lenguaje entrenado con código, que propone modificaciones *plausibles*.

Observación 1.2 (Por qué buscar programas y no soluciones). (i) Un programa *generaliza*: una heurística de empaquetado sirve para instancias nuevas, mientras que un empaquetado concreto solo sirve para la suya. (ii) Un programa corto puede *describir* un objeto enorme: en el problema del *cap set*, FunSearch buscó programas que construyen el conjunto, no el conjunto [44]. (iii) Un programa se puede *leer*: el experto puede entender, simplificar y auditar la regla encontrada. (iv) Un programa se puede *verificar*: el evaluador lo ejecuta y comprueba la validez de la salida.

Definición 1.3 (Esqueleto y función que evoluciona). Un *esqueleto* es un programa fijo que resuelve el problema llamando a una función con firma conocida, la única que se modifica durante la búsqueda. En el empaquetado en línea, el esqueleto recorre los objetos en orden de llegada, calcula los contenedores en los que cabe cada uno y coloca el objeto en el de mayor `prioridad(objeto, contenedores)`; solo `prioridad` evoluciona.

El esqueleto encierra el conocimiento del problema y garantiza la factibilidad (un objeto nunca se coloca donde no cabe), de modo que cualquier función que devuelva números produce una solución válida. Es el mismo principio de diseño que el filtro de seguridad del módulo 26 (tema 6, §6.5): la parte aprendida propone dentro de un marco que impide lo inaceptable.

1.2 FunSearch

El bucle de la figura 1.1 se ejecuta de forma asíncrona y distribuida. Según el artículo y la implementación publicada [44, 16]:

1. *Constructor del prompt.* Se elige una isla al azar y, dentro de ella, $k = 2$ programas; se ordenan de peor a mejor puntuación, se renombran como `prioridad_v0` y `prioridad_v1`, y el prompt termina con la cabecera de `prioridad_v2`, «versión mejorada de la v1». A esta técnica los autores la llaman *best-shot prompting*.
2. *Muestreo.* Un LLM preentrenado con código (en el trabajo original, un modelo de la familia PaLM 2; en la actualización de 2024, Gemini 1.5 Flash [15]) completa el cuerpo de la nueva función. La configuración por defecto de la implementación usa 15 muestreadores que piden 4 continuaciones por prompt.
3. *Evaluación.* El programa se ejecuta en un entorno aislado sobre las instancias de entrenamiento; si falla, tarda demasiado o produce algo inválido, se descarta. La configuración por defecto usa 140 evaluadores en paralelo.
4. *Base de programas.* Los programas válidos se guardan en 10 islas que evolucionan por separado. Dentro de cada isla se agrupan en *clusters* por su *firma* (el vector de puntuaciones en cada instancia de entrenamiento). Periódicamente se reinicia la mitad de las islas con peor mejor puntuación, sembrando cada una con el mejor programa de una isla superviviente elegida al azar.

Definición 1.4 (Muestreo de clusters con temperatura). Sean $s_1, \dots, s_c$ las puntuaciones de los clusters de una isla y $T > 0$ una temperatura. Se elige el cluster i con probabilidad

$$p_i(T) = \frac{\exp(s_i/T)}{\sum_{l=1}^{c} \exp(s_l/T)}.$$

En la implementación de referencia, T decrece linealmente con el número n de programas registrados en la isla, $T = T_0\big(1 - (n \bmod P)/P\big)$, con $T_0 = 0{,}1$ y periodo $P = 30\,000$; dentro del cluster elegido se prefieren los programas más cortos, con una softmax sobre la longitud normalizada cambiada de signo [16].

Proposición 1.5 (Límites de la temperatura). *Sea $M = \max_i s_i$ y $A = \{i : s_i = M\}$. Cuando $T \to 0^+$, $p_i(T) \to 1/|A|$ si $i \in A$ y $p_i(T) \to 0$ en otro caso. Cuando $T \to \infty$, $p_i(T) \to 1/c$ para todo i.*

La temperatura regula, pues, el equilibrio entre *explotar* los mejores programas y *explorar* otros que pueden ser punto de partida de mejoras mayores; es la misma distribución de

Boltzmann del recocido simulado (módulo 24, tema 5, §5.2). Las islas y los reinicios cumplen una función parecida a otra escala: mantienen poblaciones que no se contaminan entre sí y, de vez en cuando, abandonan las que se han estancado.

Proposición 1.6 (Elitismo y espera de una mejora). *Sea b_t la mejor puntuación de la base de programas tras t evaluaciones, en un algoritmo que nunca elimina el mejor programa. Entonces:*

(i) $b_{t+1} \geq b_t$ *para todo* t.

(ii) Si, mientras no haya mejora, cada propuesta supera a b_t *con probabilidad al menos* $q > 0$*, de forma independiente, el número* N *de evaluaciones hasta la primera mejora cumple* $\mathbb{E}[N] \leq 1/q$ *y* $\mathbb{P}(N > B) \leq (1-q)^B \leq e^{-qB}$.

(iii) Si I *islas independientes tienen cada una probabilidad* p *de alcanzar una puntuación objetivo con su presupuesto, la probabilidad de que la alcance alguna es* $1-(1-p)^I$.

La proposición explica el papel del LLM con una sola cifra: el presupuesto necesario es del orden de $1/q$. Una mutación aleatoria de un árbol sintáctico tiene una q minúscula; un modelo que ha visto millones de programas concentra la probabilidad en ediciones que tienen sentido (cambiar un umbral, añadir un caso, simplificar una expresión), y eso multiplica q.

1.3 AlphaEvolve

AlphaEvolve [34] generaliza FunSearch de función a sistema. El usuario marca con comentarios (`EVOLVE-BLOCK-START` y `EVOLVE-BLOCK-END`) los bloques de un fichero que pueden cambiar; el resto actúa como esqueleto. Los LLM devuelven *diferencias* en formato de búsqueda y sustitución en lugar de reescribir el fichero, lo que permite evolucionar programas de cientos de líneas en cualquier lenguaje. Un conjunto de modelos combina uno rápido (Gemini 2.0 Flash, muchas propuestas) y uno más capaz (Gemini 2.0 Pro, menos propuestas y de más calidad). La evaluación es una *cascada*: el programa pasa por pruebas de dificultad creciente y solo avanza si supera las anteriores, de modo que los candidatos malos se descartan con poco coste. La base de programas combina MAP-Elites (una población que guarda el mejor programa en cada celda de un espacio de características, para mantener diversidad) con el modelo de islas. El cuadro 1.1 resume las diferencias que los propios autores señalan.

Cuadro 1.1: FunSearch frente a AlphaEvolve, según la tabla 1 de [34].

	FunSearch	AlphaEvolve
qué evoluciona	una función	un fichero entero, con varios bloques
tamaño	10–20 líneas	hasta cientos de líneas
lenguaje	Python	cualquiera
evaluación	≤ 20 min en una CPU	horas, en paralelo, con aceleradores
muestras del LLM	millones	bastan miles

1.4 Por qué funciona y cuándo no

Tres ingredientes explican el éxito. *Asimetría entre generar y verificar*: comprobar que un empaquetado es válido y contar contenedores es inmediato, aunque encontrar una buena

heurística sea difícil; es la misma asimetría que explota el aprendizaje por refuerzo con recompensas verificables (módulo 21, tema 6, §6.2) y el esquema de generar y verificar del módulo 27 (tema 6, §6.3). *Una distribución de propuestas informada*: el LLM eleva la probabilidad q de la proposición 1.6. *Memoria y diversidad*: la base de programas acumula lo aprendido y las islas evitan la convergencia prematura.

Los límites son igual de claros. (i) *El evaluador es la especificación*: lo que no mida no se optimiza, y lo que mida mal se explota, como en la sobreoptimización de la recompensa (módulo 21, tema 6, §6.4). (ii) *Generalización*: la heurística se ajusta a la distribución de entrenamiento; Herrmann y Pallez [17] concluyen que las heurísticas de empaquetado de FunSearch no son robustas a cambios en la distribución de los datos y que en gran parte redescubren patrones conocidos. (iii) *Coste*: millones de muestras en FunSearch y horas de evaluación en AlphaEvolve. (iv) *Seguridad*: el código generado es código no confiable y debe ejecutarse aislado, sin red ni acceso a datos sensibles.

1.5 Empaquetado en línea

Definición 1.8 (Empaquetado unidimensional en línea). Se reciben, uno a uno, objetos de tamaños $s_1, s_2, \ldots, s_N \in (0, C]$ que deben colocarse en contenedores de capacidad C sin superarla. Cada objeto se coloca al llegar, sin conocer los siguientes, y no se puede mover después. El objetivo es minimizar el número de contenedores usados. Sea OPT el mínimo que se lograría conociendo todos los objetos de antemano.

Definición 1.9 (Reglas de prioridad clásicas). Sea r la capacidad restante de un contenedor abierto en el que cabe el objeto s y $g = r - s$ el hueco que dejaría. *First fit* (FF) coloca el objeto en el primer contenedor abierto en que cabe; *best fit* (BF), en el que deja menor hueco g; en ambos casos se abre uno nuevo solo si no cabe en ninguno. En términos del esqueleto, FF usa la prioridad $-$(índice del contenedor) y BF la prioridad $-g$. Un algoritmo es de tipo *any fit* si nunca abre un contenedor nuevo cuando el objeto cabe en uno abierto.

Lema 1.10 (Cota L_1). $\mathrm{OPT} \geq L_1 = \lceil \sum_j s_j / C \rceil$.

Teorema 1.11 (Garantía de los algoritmos *any fit*). *Todo algoritmo* any fit*, en particular FF y BF, usa a lo sumo* $\lceil 2 \sum_j s_j / C \rceil \leq 2\,\mathrm{OPT}$ *contenedores.*

La constante 2 no es la mejor posible para FF, pero el resultado ilustra el tipo de garantía que una heurística generada *no* trae consigo: lo que se sabe de ella es lo que mide el evaluador. Para el problema sin restricciones de llegada, Williamson y Shmoys demuestran que, salvo que P = NP, ningún algoritmo de tiempo polinómico garantiza un factor menor que 3/2 [50, §3.3].

1.6 Caso de estudio: una heurística de empaquetado generada

Como referencia externa, en el conjunto Weibull de 5000 objetos que acompaña al artículo, BF usa de media 2067,0 bandejas y la heurística de FunSearch 2001,4, con $\bar{L}_1 = 1987{,}8$: excesos del 3,98 % y del 0,68 % [16]. La *prueba de robustez* del cuaderno repite la comparación con otra distribución, Weibull de escala 60 (objetos más grandes): BF deja un exceso del 5,19 %, la regla π^* un 5,21 % y la heurística publicada un 6,40 %, peor que FF (6,05 %). Con seis semillas distintas del generador, el exceso en prueba de la regla encontrada varía entre el

Cuadro 1.2: Empaquetado en línea de 5 instancias de prueba de 5000 objetos ($\bar{L}_1 = 1985{,}6$). Datos del cuaderno.

Regla	Bandejas (media)	Exceso sobre L_1
first fit	2073,4	4,42 %
best fit	2064,8	3,99 %
regla evolucionada π^*	2011,2	1,29 %
heurística publicada por FunSearch	2000,0	0,73 %

1,29 % y el 2,33 %, y cuatro de las seis reglas tienen la misma estructura: un término de *best fit* saturado más una penalización de los huecos intermedios. Las dos heurísticas generadas se han especializado en la distribución con la que se evaluaron, tal como advierten Herrmann y Pallez [17].

Cuadro 1.3: Tecnologías del tema 1.

Herramienta	Para qué sirve	Licencia o tipo	Cuándo elegirla
FunSearch (repositorio) [16]	algoritmo evolutivo, manipulación de código y una versión de un solo hilo, sin LLM ni entorno aislado	abierta (Apache 2.0)	entender el método y adaptarlo a un LLM y una infraestructura propios
OpenEvolve [35]	implementación abierta inspirada en AlphaEvolve: islas, MAP-Elites, evaluación en cascada, conjuntos de LLM	abierta (Apache 2.0)	evolucionar ficheros completos con cualquier API compatible
AlphaEvolve [34]	agente de Google DeepMind	servicio, no publicado como código	referencia de diseño; resultados publicados
gplearn, DEAP	programación genética clásica sin LLM	abiertas (BSD-3; LGPL)	línea base barata para expresiones pequeñas
Contenedores aislados	ejecutar código no confiable sin red y con límites de tiempo y memoria	abierta o del proveedor	siempre que se ejecute código generado

Tema 2

Optimización de instrucciones, de sistemas con LLM y de código

Un sistema construido con modelos de lenguaje tiene muchos parámetros que no son números: las instrucciones de cada llamada, los ejemplos que se incluyen, el orden de los pasos, el número de muestras que se votan. Este tema los trata como variables de decisión de un problema de optimización de caja negra con un objetivo medido sobre datos. Sigue a Yang *et al.* para OPRO [51], a Khattab *et al.* para DSPy [22] y a Opsahl-Ong *et al.* para MIPRO [36]; la autoconsistencia, a Wang *et al.* [49] y al módulo 22 (tema 1, §1.4); el sesgo del ganador, a Smith y Winkler [47]; la optimización de código y la medida de tiempos, a la clase 10 «Measurement and Timing» del curso 6.172 del MIT [32].

2.1 Instrucciones como variables de decisión

Definición 2.1 (Optimización de instrucciones). Sea $\mathcal{I}$ un conjunto de instrucciones (textos), M un modelo de lenguaje fijo, $D_{\text{val}} = \{(x_i, y_i)\}_{i=1}^{n}$ un conjunto de validación y $m(\hat{y}, y) \in [0, 1]$ una métrica (por ejemplo, $\mathbf{1}[\hat{y} = y]$). El problema es

$$\max_{p \in \mathcal{I}} J(p), \qquad J(p) = \mathbb{E}_{(x,y)}\big[m(M(p, x), y)\big], \qquad \hat{J}(p) = \frac{1}{n}\sum_{i=1}^{n} m\big(M(p, x_i), y_i\big).$$

Es un problema de caja negra (módulo 24, definición 5.1), discreto, ruidoso (el estimador $\hat{J}$ tiene varianza $J(1-J)/n$ si m es binaria) y caro (cada evaluación son n llamadas al modelo).

Definición 2.2 (OPRO). *Optimization by PROmpting* [51] usa un segundo LLM como optimizador. En cada paso se le da un *meta-prompt* con: una descripción de la tarea y unos pocos ejemplos, y la *trayectoria*: las mejores instrucciones encontradas hasta el momento con su puntuación, ordenadas de menor a mayor. El LLM optimizador genera nuevas instrucciones; se evalúan con el LLM que resuelve la tarea, se añaden a la trayectoria y se repite. En los experimentos del artículo, la trayectoria conserva las 20 mejores, se generan 8 instrucciones por paso con temperatura 1 y la puntuación es la exactitud sobre un subconjunto fijo del conjunto de entrenamiento.

OPRO es una búsqueda evolutiva como la del tema 1, con dos diferencias: el «programa» es un texto y la memoria es la propia trayectoria, que el LLM lee para inferir qué rasgos se asocian a puntuaciones altas. Los autores informan de que las mejores instrucciones encontradas superan a las diseñadas por personas hasta en 8 puntos en GSM8K y hasta en 50 en tareas de Big-Bench Hard [51]; también advierten de que, en el problema del viajante, el LLM se mantiene cerca de las heurísticas clásicas con 50 ciudades, pero en instancias mayores la inserción más lejana supera a todos los modelos probados.

Definición 2.3 (Programa de modelos de lenguaje y su compilación). En DSPy [22], un *programa de LM* es un grafo de *módulos*; cada módulo tiene una *firma* declarativa (entradas y salidas con nombre, por ejemplo `pregunta, contexto -> respuesta`) y unos parámetros: su instrucción y sus demostraciones. Un *optimizador* (antes llamado *teleprompter*) recibe el programa, un conjunto de entrenamiento y una métrica, y elige los parámetros de todos los módulos para maximizar la métrica del programa completo. Por ejemplo, el arranque de demostraciones ejecuta el programa sobre los ejemplos de entrenamiento y conserva como demostraciones las trazas que superan la métrica.

Khattab *et al.* informan de que los programas compilados superan a los mismos programas con pocos ejemplos estándar en más de un 25 % con GPT-3.5 y un 65 % con llama2-13b-chat [22]. MIPRO [36] optimiza conjuntamente instrucciones y demostraciones de cada módulo sin etiquetas intermedias: propone instrucciones teniendo en cuenta los datos y el programa, y usa evaluaciones en minilotes para ajustar un modelo sustituto del objetivo, como la optimización bayesiana del módulo 25 (tema 4); obtiene mejoras de hasta 13 puntos de exactitud con Llama-3-8B.

2.2 El sesgo del ganador

Todas estas técnicas eligen el máximo de muchas estimaciones ruidosas. El resultado es predecible.

Proposición 2.4 (Maldición del optimizador). *Sean K candidatos con valores verdaderos $J_1, \dots, J_K$ y estimaciones $\hat{J}_k = J_k + \varepsilon_k$ con $\mathbb{E}[\varepsilon_k] = 0$, y sea $\hat{k} = \arg\max_k \hat{J}_k$. Entonces:*

(i) $\mathbb{E}[\hat{J}_{\hat{k}}] \geq \max_k J_k \geq \mathbb{E}[J_{\hat{k}}]$*: la puntuación de validación del elegido sobrestima en media su valor verdadero.*

(ii) Si cada $\varepsilon_k \sim \mathcal{N}(0, \sigma^2)$ (no necesariamente independientes), $\mathbb{E}[\max_k \varepsilon_k] \leq \sigma\sqrt{2 \ln K}$. En particular, si todos los J_k son iguales, el sesgo $\mathbb{E}[\hat{J}_{\hat{k}} - J_{\hat{k}}]$ está entre 0 y $\sigma\sqrt{2 \ln K}$.

Corolario 2.6 (Protocolo de evaluación). *Para estimar sin sesgo el valor de un sistema optimizado se necesita un conjunto de* prueba *que no haya intervenido en ninguna elección (ni de instrucciones, ni de ejemplos, ni de hiperparámetros), igual que en la separación en entrenamiento, validación y prueba del módulo 8 (tema 1, §1.4). Su error típico, $\sqrt{J(1-J)/n_{\text{test}}}$, fija la resolución con la que se pueden comparar sistemas.*

2.3 Caso de estudio: clasificación de partes de avería

2.4 La autoconsistencia como problema de optimización

La autoconsistencia (módulo 22, definición 1.14) devuelve la respuesta más votada entre m muestras. Más muestras dan más exactitud y más coste; el número de muestras es una variable de decisión.

Proposición 2.8 (Incremento de la exactitud del voto). *Sea P_m la probabilidad de que la mayoría de m (impar) respuestas independientes sea correcta cuando cada una lo es con*

probabilidad $p > 1/2$ (módulo 22, proposición 1.16). Entonces

$$\Delta_m := P_{m+2} - P_m = \binom{m}{\frac{m-1}{2}} \big(p(1-p)\big)^{\frac{m+1}{2}} (2p-1) > 0, \qquad \frac{\Delta_{m+2}}{\Delta_m} = \frac{4(m+2)}{m+3}\, p(1-p) < 1.$$

Corolario 2.9 (Número óptimo de muestras). *Si una respuesta correcta vale V y cada muestra cuesta c, el beneficio neto $VP_m - cm$ (con m impar) se maximiza en el primer m tal que $V\Delta_m < 2c$.*

Con $p = 0{,}65$, $V = 0{,}50$ € y $c = 0{,}01$ €: $\Delta_1 = 0{,}0683$, $\Delta_3 = 0{,}0466$ y $\Delta_5 = 0{,}0353$; $V\Delta_3 = 0{,}0233 \geq 0{,}02$ pero $V\Delta_5 = 0{,}0177 < 0{,}02$, de modo que $m^* = 5$, con exactitud $P_5 = 0{,}765$ y beneficio neto 0,332 € por parte. La hipótesis de independencia es optimista (módulo 22, ejemplo 1.17): conviene estimar P_m con datos y aplicar el mismo criterio.

2.5 Optimización de código con agentes

Un agente de programación (módulo 22, tema 6) puede usarse como optimizador de un programa existente. El planteamiento correcto es el de un problema con restricciones:

$$\min_{\pi}\ t(\pi) \quad \text{s.a.} \quad \pi \text{ supera todas las pruebas}, \quad \text{memoria}(\pi) \leq \bar{M}, \quad \text{coste del agente} \leq B,$$

donde t es el tiempo de ejecución en una carga representativa. Las pruebas son una *restricción dura*, no un término del objetivo: un programa más rápido que falla una prueba no es una solución. Es el mismo principio del evaluador del tema 1.

Proposición 2.10 (Ley de Amdahl). *Si una fracción f del tiempo de ejecución se acelera en un factor $a > 1$ y el resto no cambia, la aceleración global es*

$$A = \frac{1}{(1-f) + f/a} < \frac{1}{1-f}.$$

La ley dice dónde empezar: por lo que el *perfilador* muestra que ocupa más tiempo. Un agente que no perfila tiende a optimizar lo que parece lento en el código, no lo que lo es.

Cuadro 2.1: Tecnologías del tema 2.

Herramienta	Para qué sirve	Licencia o tipo	Cuándo elegirla
DSPy [22]	programas de LM declarativos y optimizadores de instrucciones y demostraciones (incluido MIPRO)	abierta (MIT)	sistemas de varias llamadas con una métrica clara
OPRO [51]	método: un LLM como optimizador a partir de la trayectoria	artículo con código de los autores	instrucciones de una sola llamada; problemas pequeños
promptfoo	conjuntos de pruebas y comparación de instrucciones y modelos	abierta (MIT)	evaluación de regresión de instrucciones
pytest, pytest-benchmark	pruebas como restricción y medidas de tiempo repetidas	abiertas (MIT; BSD-2)	optimización de código en Python
perf, py-spy, Scalene	perfilado de CPU y memoria	abiertas (GPL-2.0; MIT; Apache 2.0)	antes de pedir al agente que optimice nada
asv	seguimiento del rendimiento a lo largo del historial del código	abierta (BSD-3)	evitar regresiones de rendimiento

Tema 3

AutoML y búsqueda de arquitecturas

Elegir un modelo, sus hiperparámetros y su preprocesado es también un problema de optimización sobre programas: cada configuración es una canalización ejecutable cuyo valor se mide con validación. Este tema formaliza ese problema siguiendo el libro editado por Hutter, Kotthoff y Vanschoren [19] (cap. 1 «Hyperparameter Optimization» y cap. 3 «Neural Architecture Search», de Elsken, Metzen y Hutter [12]) y los artículos originales: Auto-WEKA [48], búsqueda aleatoria [3], búsqueda de arquitecturas con refuerzo [53], con evolución [41], con pesos compartidos [38] y diferenciable [28], y la línea base de Li y Talwalkar [26]. La multifidelidad sigue a Jamieson y Talwalkar [21], a Li *et al.* [27] y a la §19.4 de Zhang *et al.* [52]. Complementa el tema 6 del módulo 25, que estudió el ajuste de hiperparámetros con optimización bayesiana y Hyperband; aquí el énfasis está en el espacio de búsqueda y en el coste.

3.1 AutoML como optimización sobre programas

Definición 3.1 (Selección conjunta de algoritmo e hiperparámetros). Se consideran R algoritmos de aprendizaje, $A^{(1)}, \dots, A^{(R)}$, cada uno con un espacio de hiperparámetros $\Lambda^{(j)}$, y sea D un conjunto de datos dividido en k pares $(D_{\text{tr}}^{(i)}, D_{\text{va}}^{(i)})$, cada uno formado por una parte de entrenamiento y otra de validación. El problema CASH (*combined algorithm selection and hyperparameter optimization*) [48] es

$$(j^*, \lambda^*) \in \operatorname*{arg\,min}_{j \in \{1,\dots,R\},\ \lambda \in \Lambda^{(j)}} \frac{1}{k} \sum_{i=1}^{k} \mathcal{L}\big(A_\lambda^{(j)}, D_{\text{tr}}^{(i)}, D_{\text{va}}^{(i)}\big),$$

donde $\mathcal{L}$ es la pérdida de validación del modelo entrenado con $D_{\text{tr}}^{(i)}$.

El espacio de la definición 3.1 es *condicional*: la profundidad de un árbol solo existe si se elige un árbol. Se representa como un único espacio jerárquico cuya raíz es el hiperparámetro «algoritmo», de modo que métodos como la optimización bayesiana con bosques aleatorios (SMAC) o con estimadores de densidad en árbol (TPE) lo tratan sin cambios. Si además se eligen el preprocesado, la selección de variables y la combinación de modelos, la configuración es un programa completo; TPOT, por ejemplo, busca esas canalizaciones con programación genética, como en el tema 1 [19].

Observación 3.2 (Por qué la búsqueda aleatoria es una línea base seria). Si la pérdida depende sobre todo de unos pocos hiperparámetros, una rejilla desperdicia evaluaciones repitiendo los mismos valores de los importantes, mientras que n puntos aleatorios prueban n valores distintos de cada uno [3]. Además, la probabilidad de que al menos uno de n puntos aleatorios caiga en la fracción q mejor del espacio es $1 - (1-q)^n$, independiente de la dimensión: con $q = 0{,}05$ y $n = 60$ vale 0,954 (módulo 25, tema 6).

3.2 Búsqueda de arquitecturas neuronales

Elsken, Metzen y Hutter organizan la búsqueda de arquitecturas (NAS) en tres dimensiones [12]: el *espacio de búsqueda* (qué arquitecturas pueden representarse), la *estrategia de búsqueda* (cómo se explora) y la *estimación del rendimiento* (cómo se evalúa una arquitectura sin entrenarla del todo).

Definición 3.3 (Espacio de celdas). Una *celda* es un grafo dirigido acíclico con dos nodos de entrada (las salidas de las dos celdas anteriores) y B nodos intermedios. El nodo intermedio b elige dos entradas entre los $b+1$ nodos anteriores y aplica a cada una una operación de un conjunto $\mathcal{O}$ (convoluciones, agrupamientos, identidad…); su salida es la suma de ambas. La red se construye apilando copias de una celda normal y de una celda de reducción.

Si las dos elecciones de cada nodo son ordenadas y pueden repetirse, el número de celdas distintas es $\prod_{b=1}^{B}\big((b+1)|\mathcal{O}|\big)^2$: con $B=5$ y $|\mathcal{O}|=8$, unas $5{,}6\cdot 10^{14}$. La celda reduce el espacio respecto a una red arbitraria, pero sigue siendo inabarcable por enumeración.

Estrategias de búsqueda. Zoph y Le entrenaron con refuerzo una red recurrente que genera descripciones de arquitecturas y obtuvieron un 3,65 % de error en CIFAR-10 [53]. Real *et al.* usaron evolución por torneo con *envejecimiento* (se elimina el individuo más antiguo, no el peor), y en una comparación controlada con el mismo hardware la evolución llegó antes a buenos resultados que el refuerzo [41]. Liu, Simonyan y Yang hicieron el espacio continuo:

Definición 3.4 (Relajación continua de DARTS). En DARTS [28], cada arista (i,j) de la celda aplica una *mezcla* de todas las operaciones,

$$\bar{o}^{(i,j)}(x)=\sum_{o\in\mathcal{O}}\frac{\exp\big(\alpha_o^{(i,j)}\big)}{\sum_{o'\in\mathcal{O}}\exp\big(\alpha_{o'}^{(i,j)}\big)}\,o(x),$$

y los parámetros de arquitectura α se optimizan por descenso de gradiente en un problema binivel: $\min_\alpha \mathcal{L}_{\text{va}}\big(w^*(\alpha),\alpha\big)$ con $w^*(\alpha)=\arg\min_w \mathcal{L}_{\text{tr}}(w,\alpha)$. Al terminar, cada arista conserva la operación de mayor α.

Estimación del rendimiento. Entrenar cada candidata hasta el final es lo que hace prohibitiva la búsqueda. ENAS comparte los pesos entre todas las arquitecturas del espacio (cada candidata es un subgrafo de un modelo grande) y lo abarata en unas mil veces en horas de GPU respecto a la búsqueda original [38]. Li y Talwalkar mostraron que la búsqueda aleatoria con parada temprana iguala a ENAS en PTB y CIFAR-10, y con pesos compartidos lo supera, y señalaron problemas de reproducibilidad en la literatura [26]. La lección práctica es la del tema 1: compárese siempre con una búsqueda aleatoria del mismo presupuesto.

3.3 Multifidelidad y coste

Definición 3.5 (Fidelidad). Una *fidelidad* es un recurso $r\in[r_{\min},r_{\max}]$ que controla el coste y la precisión de la evaluación de una configuración λ: épocas de entrenamiento, fracción de los datos, resolución de las imágenes o número de árboles. Se escribe $f(\lambda,r)$ para la pérdida de validación con presupuesto r; la de interés es $f(\lambda,r_{\max})$.

Definición 3.6 (Halving sucesivo). Dados N configuraciones, un factor $\eta\geq 2$ y un presupuesto mínimo $r_{\min}$, el *halving sucesivo* [21] evalúa todas con $r_0=r_{\min}$, conserva la fracción

$1/\eta$ mejor, multiplica el presupuesto por η y repite: en el escalón i quedan $n_i = N/\eta^i$ configuraciones con presupuesto $r_i = r_{\min}\eta^i$, hasta $r_K = r_{\max}$.

Proposición 3.7 (Coste del halving sucesivo). *Si N es múltiplo de η^K y el coste de una evaluación es proporcional a su presupuesto, cada escalón cuesta $Nr_{\min}$ y el total es $(K+1)Nr_{\min}$, frente a $Nr_{\max} = Nr_{\min}\eta^K$ de evaluar todas con el presupuesto completo. El cociente es $(K+1)/\eta^K$.*

Proposición 3.8 (Cuándo acierta el halving sucesivo). *Sea $\lambda^\dagger$ la mejor configuración a fidelidad completa. Si en cada escalón $i < K$ la ordenación por $f(\cdot, r_i)$ coloca a $\lambda^\dagger$ entre las n_{i+1} primeras de las que quedan, el halving sucesivo devuelve $\lambda^\dagger$. Si las curvas de aprendizaje se cruzan (una configuración peor al principio es mejor al final), puede descartarla.*

La hipótesis de la proposición 3.8 se puede comprobar en un piloto con la correlación de rangos de Spearman entre las pérdidas a baja y a alta fidelidad. Hyperband [27] protege frente a una mala elección de $r_{\min}$ ejecutando varias rondas de halving sucesivo con distinta agresividad (módulo 25, tema 6). En la práctica, el coste no es exactamente proporcional al presupuesto: cargar los datos, compilar el modelo o arrancar una máquina son costes fijos que reducen el ahorro de la proposición 3.7.

3.4 Caso de estudio: AutoML del clasificador de averías

Cuadro 3.1: Tecnologías del tema 3.

Herramienta	Para qué sirve	Licencia o tipo	Cuándo elegirla
Optuna [1]	optimización de hiperparámetros con espacios definidos en el propio código (TPE y otros) y poda con halving sucesivo o Hyperband	abierta (MIT)	ajuste de cualquier modelo en Python con poco código
Ray Tune [40]	ajuste distribuido en un clúster con planificadores de multifidelidad	abierta (Apache 2.0)	muchas pruebas en paralelo sobre varias máquinas
SMAC3	configuración algorítmica con bosques aleatorios como sustituto	abierta (BSD-3)	espacios condicionales, configurar resolvedores
auto-sklearn, AutoGluon	AutoML completo para datos tabulares (selección, ajuste, conjuntos)	abiertas (BSD-3; Apache 2.0)	primera línea base fuerte en datos tabulares
NNI	búsqueda de arquitecturas y de hiperparámetros	abierta (MIT)	NAS con estrategias variadas sobre PyTorch

Tema 4

Del prototipo a la producción: arquitectura, MLOps y evaluación

Un cuaderno que predice averías y propone un calendario es un prototipo; un sistema que lo hace cada día durante años, con datos que cambian, usuarios que dependen de él y auditores que preguntan por qué decidió algo, es otra cosa. Este tema describe esa otra cosa. La arquitectura y el ciclo de vida siguen el curso CS329S de Stanford [46] y el libro de Huyen [18]; la deuda técnica, a Sculley *et al.* [45]; la lista de pruebas, a Breck *et al.* [5]; la evaluación contrafactual, a Dudík, Langford y Li [11]; la fiabilidad, al capítulo 4 «Optimum Maintenance Policies» de Barlow y Proschan [2] y al curso 1.010 del MIT [29]. Completa la arquitectura del módulo 24 (tema 1, §1.2) y la puesta en planta del módulo 26 (tema 6), que trató el control por capas, los modos de operación, las alarmas y la ciberseguridad.

4.1 Arquitectura de un optimizador en producción

La figura 4.1 separa responsabilidades que en el prototipo estaban mezcladas.

1. *Datos*: la ingesta valida un *esquema* (tipos, rangos, unidades, frecuencias, ausencias) antes de aceptar nada; los datos y las características se *versionan*, de modo que se pueda reconstruir exactamente con qué datos se entrenó cada modelo.
2. *Características*: se calculan con el mismo código al entrenar y al servir, y respetando el tiempo: cada fila de entrenamiento solo puede usar información disponible en su instante.
3. *Modelos*: se entrenan con una canalización reproducible y se guardan en un *registro* con sus métricas, sus datos de origen y su estado (candidato, en sombra, en producción, retirado).
4. *Servicio*: una API con un contrato explícito (entradas, salidas, unidades, versión del modelo, latencia máxima) desacopla a los consumidores del modelo concreto.
5. *Optimizador*: combina las predicciones con el modelo de decisión y un resolvedor (módulo 24); sus restricciones duras no dependen del modelo aprendido.
6. *Interfaz humana*: presenta la propuesta con sus motivos, permite aprobarla, modificarla o anularla, y registra la decisión final; es el lugar donde se ejerce la supervisión que exige la regulación (tema 5).
7. *Monitorización*: vigila los datos, las predicciones, las decisiones, los indicadores de negocio y la salud del servicio, y dispara acciones definidas de antemano.

Definición 4.1 (Sesgo entre entrenamiento y servicio). Hay *sesgo entre entrenamiento y servicio* cuando la distribución de las entradas que el modelo recibe en producción difiere de la de entrenamiento por causas del propio sistema y no del mundo: características calculadas

con código distinto, unidades distintas, datos que en entrenamiento estaban disponibles y en servicio llegan tarde, o filas de entrenamiento que contienen información del futuro.

4.2 Deuda técnica en sistemas con aprendizaje automático

Sculley *et al.* [45] observan que en un sistema real el código del modelo es una pequeña parte del total y enumeran riesgos propios de estos sistemas, que generan *deuda técnica*: costes futuros que se contraen al tomar atajos hoy.

Definición 4.2 (Factores de riesgo de Sculley *et al.*). (a) *Erosión de fronteras y entrelazamiento*: en un modelo, cambiar cualquier entrada cambia todo lo demás (principio CACE, *changing anything changes everything*); no hay módulos independientes.

(b) *Bucles de realimentación ocultos*: las decisiones del sistema cambian los datos con que se reentrena.

(c) *Consumidores no declarados*: otros sistemas usan las salidas del modelo sin que nadie lo sepa, y se rompen al cambiarlo.

(d) *Dependencias de datos*: entradas inestables (su significado cambia) o infrautilizadas (aportan poco y añaden riesgo).

(e) *Antipatrones de sistema*: código pegamento, junglas de canalizaciones, rutas experimentales muertas.

(f) *Deuda de configuración*: miles de parámetros de configuración sin revisión ni pruebas.

(g) *Cambios en el mundo exterior*: umbrales fijos que dejan de valer, correlaciones que desaparecen.

Observación 4.3 (El bucle oculto del mantenimiento predictivo). Si el sistema adelanta las intervenciones de las máquinas con riesgo alto, esas máquinas fallarán menos, y los datos futuros mostrarán pocos fallos asociados a vibraciones altas, porque se intervino antes. Un modelo reentrenado ingenuamente con esos datos concluirá que la vibración alta es menos peligrosa y relajará las intervenciones. Es el bucle de realimentación degenerado de la clase 10 de CS329S [46]. Remedios: registrar cada decisión con su probabilidad (apartado 4.4), tratar las intervenciones como censura en el modelo de supervivencia y mantener una pequeña fracción de decisiones exploratorias cuando sea seguro.

Breck *et al.* [5] convierten estos riesgos en una lista de 28 pruebas agrupadas en cuatro bloques: datos y características, desarrollo del modelo, infraestructura y monitorización; cada prueba manual puntúa medio punto y cada prueba automatizada, uno. Es una buena lista de comprobación antes de desplegar.

4.3 Deriva y reentrenamiento

Definición 4.4 (Tipos de cambio de distribución). Sea X la entrada y Y la salida, con $P(X,Y) = P(Y \mid X)P(X) = P(X \mid Y)P(Y)$. Siguiendo la clase 10 de CS329S [46], hay *cambio de covariables* si cambia $P(X)$ y no $P(Y \mid X)$; *cambio de etiquetas* si cambia $P(Y)$ y no $P(X \mid Y)$; y *deriva de concepto* si cambia $P(Y \mid X)$. Además pueden cambiar las propias características (unidades, definición) o el esquema de las etiquetas.

Definición 4.5 (Índice de estabilidad poblacional). Sea una variable discretizada en B intervalos, con proporciones p_b en la referencia (entrenamiento) y q_b en una ventana reciente, todas positivas. El *índice de estabilidad poblacional* es

$$\mathrm{PSI}(p, q) = \sum_{b=1}^{B} (q_b - p_b) \ln \frac{q_b}{p_b}.$$

Proposición 4.6 (El PSI es la divergencia simetrizada). $\mathrm{PSI}(p, q) = \mathrm{KL}(p \,\|\, q) + \mathrm{KL}(q \,\|\, p) \geq 0$, *con igualdad si y solo si* $p = q$.

Las métricas de exactitud son la señal más directa de degradación, pero sus etiquetas llegan con retraso (en el caso de estudio, 7 días). Mientras tanto se vigilan las entradas (PSI, contraste de Kolmogorov–Smirnov por variable) y la distribución de las predicciones, más barata por ser de baja dimensión [46]. Muchas alarmas de deriva resultan ser errores internos: valores ausentes, cambios de unidades, canalizaciones rotas.

Proposición 4.8 (Periodo óptimo de reentrenamiento). *Supóngase que, tras reentrenar, la pérdida diaria por degradación del modelo crece linealmente, at (€/día) con* $a > 0$, *y que cada reentrenamiento (cómputo, validación, despliegue) cuesta* c_r. *Con un periodo fijo* T, *el coste medio por día es* $c_r/T + aT/2$, *que se minimiza en* $T^* = \sqrt{2c_r/a}$ *con coste* $\sqrt{2ac_r}$.

Es la fórmula del lote económico aplicada al modelo. En la práctica se combina con disparadores: se reentrena antes si la monitorización detecta una deriva o una caída de las métricas, y nunca se despliega un modelo reentrenado sin pasar las mismas validaciones que el primero.

4.4 Evaluar decisiones sin experimentar

Para saber si una política nueva es mejor que la actual hay tres caminos: un experimento en la planta (tema 5), un simulador (el gemelo digital del módulo 26, tema 2) o los registros históricos. Los registros solo muestran el resultado de la decisión que se tomó; el de las demás es *contrafactual*.

Proposición 4.9 (Estimador por ponderación inversa de la propensión). *Sean registros* (x_i, a_i, r_i), $i = 1, \ldots, n$, *independientes, donde el contexto* x_i *sigue una distribución fija, la acción* a_i *se eligió con una política de registro conocida* $\pi_0(a \mid x)$ *y* r_i *es la recompensa observada. Sea* π *una política nueva con* $\pi(a \mid x) > 0 \Rightarrow \pi_0(a \mid x) > 0$. *Entonces*

$$\hat{V}_{\mathrm{IPS}}(\pi) = \frac{1}{n} \sum_{i=1}^{n} \frac{\pi(a_i \mid x_i)}{\pi_0(a_i \mid x_i)} \, r_i$$

es un estimador insesgado del valor $V(\pi) = \mathbb{E}_x \mathbb{E}_{a \sim \pi(\cdot \mid x)} \big[\mathbb{E}[r \mid x, a] \big]$.

La varianza crece con los cocientes π/π_0: si la política antigua casi nunca tomaba la acción que la nueva prefiere, la estimación es inútil. El estimador doblemente robusto de Dudík *et al.* [11] combina un modelo de la recompensa con la ponderación y es consistente si cualquiera de los dos es correcto. La consecuencia de diseño es sencilla: *registrar siempre la acción, el contexto y la probabilidad con que se eligió*; un sistema determinista que no explora nunca permite evaluar contrafactuales con sus propios datos.

4.5 Fiabilidad y mantenimiento

Definición 4.10 (Fiabilidad y tasa de fallo). Sea $T \geq 0$ la vida útil de un componente, con función de distribución F y densidad f. La *fiabilidad* es $R(t) = 1 - F(t)$ y la *tasa de fallo* es $\lambda(t) = f(t)/R(t)$: $\lambda(t)\,dt$ es la probabilidad de fallar en $(t, t+dt)$ habiendo sobrevivido hasta t. Se cumple $R(t) = \exp\bigl(-\int_0^t \lambda(u)\,du\bigr)$ [29]. La vida es de tasa creciente (IFR) si λ es no decreciente y de tasa decreciente (DFR) si es no creciente.

Definición 4.11 (Distribución de Weibull). T sigue una Weibull de forma $\beta > 0$ y escala $\theta > 0$ si $R(t) = \exp\bigl(-(t/\theta)^{\beta}\bigr)$. Su tasa de fallo es $\lambda(t) = \frac{\beta}{\theta}\bigl(\frac{t}{\theta}\bigr)^{\beta-1}$: decreciente si $\beta < 1$ (rodaje), constante si $\beta = 1$ (exponencial, sin memoria) y creciente si $\beta > 1$ (desgaste). Su media es $\theta\,\Gamma(1 + 1/\beta)$.

Observación 4.12 (Censura). En los históricos de mantenimiento, muchas vidas no terminan en fallo: la máquina se intervino antes. Esas observaciones están *censuradas por la derecha*: solo se sabe que $T > t_c$. La verosimilitud correcta es $\prod_{\text{fallos}} f(t_i) \prod_{\text{censuras}} R(t_c)$. Descartar las censuras, o tratarlas como fallos, sesga la estimación porque elimina precisamente las vidas largas.

Proposición 4.13 (Política de sustitución por edad). *Bajo la política «intervenir preventivamente al alcanzar la edad T, o correctivamente al fallar antes», con costes $c_p < c_f$ y renovación completa tras cada intervención, el coste medio a largo plazo por unidad de tiempo es*

$$C(T) = \frac{c_p R(T) + c_f F(T)}{\int_0^T R(u)\,du}.$$

Sea $G(T) = \lambda(T)\int_0^T R(u)\,du - F(T)$. Entonces $C'(T)$ tiene el signo de $G(T) - c_p/(c_f - c_p)$, y $G'(T) = \lambda'(T)\int_0^T R(u)\,du$. En consecuencia:

(i) si la vida es DFR (en particular, Weibull con $\beta \leq 1$), C es decreciente y lo óptimo es no intervenir preventivamente ($T^* = \infty$);

(ii) si λ es creciente y $\lambda(T) \to \infty$ (Weibull con $\beta > 1$), existe un único T^* con $G(T^*) = c_p/(c_f - c_p)$, y en él $C(T^*) = (c_f - c_p)\lambda(T^*)$.

La política por edad usa solo el tiempo. El mantenimiento *predictivo* usa además el estado medido de cada máquina: un clasificador estima la probabilidad de fallo en el horizonte y la decisión se toma con esa probabilidad, como en la decisión con predicciones del módulo 24 (tema 6, §6.1). Si la probabilidad de fallo en los próximos 7 días es p_7, intervenir hoy compensa frente a no intervenir en la semana cuando $c_p < c_f p_7$, es decir, cuando $p_7 > c_p/c_f$; las restricciones de capacidad y de calendario convierten esa regla en un problema de asignación.

4.6 Caso de estudio: mantenimiento predictivo de una línea de envasado

Con un horizonte de una semana (cuadro 4.1), la llenadora, la taponadora y la encajonadora preferirían el lunes, pero solo hay un técnico. El ahorro respecto a no intervenir es $3780 - 1200 = 2580$ para la llenadora el lunes y $2790 - 2105 = 685$ para la encajonadora el miércoles; la alternativa (encajonadora el lunes y llenadora el miércoles) ahorra $1590 + 1283 = 2873 < 3265$. El coste óptimo es $10\,170 - 3265 = 6905$ €. Con el horizonte de dos semanas, el modelo añade

Cuadro 4.1: Costes de cada opción en un lunes con $p_7 = (0{,}42;\ 0{,}18;\ 0{,}07;\ 0{,}31;\ 0{,}03;\ 0{,}12)$ y horizonte de una semana (en €). En negrita, la solución óptima con un técnico por ventana.

	llenadora	taponadora	etiquetadora	encajonadora	paletizadora	enfardadora
λ_i (1/día)	0,0778	0,0284	0,0104	0,0530	0,0044	0,0183
lunes ($d = 0$)	**1200**	1200	1200	1200	1200	1200
miércoles ($d = 2$)	2497	1696	1385	**2105**	1278	1523
viernes ($d = 4$)	3607	2165	1566	2920	1355	1834
no intervenir	3780	**1620**	**630**	2790	**270**	**1080**

además la taponadora el viernes, porque no intervenirla en catorce días es más arriesgado que en siete.

Cuadro 4.2: Un año de operación de la línea en el gemelo, con los mismos números aleatorios para todas las políticas (módulo 26, tema 5, §5.1).

Política	Coste (€/año)	Averías	Preventivos	Fuera de ventana
A. Solo correctivo	360 000	40	0	—
H. Actual: cada 70 días	327 400	35	7	5
B. Por edad, $T^* = 30$ días	195 200	10	65	34
C. Umbral $p_7 \geq 0{,}10$, sin calendario	173 200	6	70	44
D. Calendario con riesgo (PLEM)	141 000	7	65	0

El cuadro 4.2 resume la simulación del cuaderno. El umbral de la política C (0,10, cercano a c_p/c_f) se eligió en un año de validación distinto. La política C, además de intervenir fuera de ventana, viola la capacidad del técnico cinco días. La política D respeta todas las restricciones y es la más barata. Que la mejora frente a B sea real, y no suerte de este año, es la pregunta del tema 5.

Cuadro 4.3: Tecnologías del tema 4.

Herramienta	Para qué sirve	Licencia o tipo	Cuándo elegirla
DVC [7]	versionar datos y modelos junto al código, con canalizaciones reproducibles	abierta (Apache 2.0)	equipos pequeños que ya usan git
MLflow [10]	seguimiento de experimentos, registro de modelos y empaquetado	abierta (Apache 2.0)	trazabilidad de qué modelo está en producción
Great Expectations, Feast	validación de esquemas y almacén de características	abiertas (Apache 2.0)	evitar el sesgo de la definición 4.1
Evidently [8]	informes y monitorización de deriva de datos y predicciones	abierta (Apache 2.0)	vigilancia periódica con pocos recursos
FastAPI [9]	servicio de predicción y optimización como API HTTP con esquema	abierta (MIT)	exponer el modelo y el resolvedor a otros sistemas
Kubernetes [25], Ray Serve [40]	despliegue en contenedores, escalado, versiones en paralelo	abiertas (Apache 2.0)	varios servicios con requisitos de disponibilidad
lifelines, scikit-survival	modelos de supervivencia con censura	abiertas (MIT; GPL-3.0)	estimar fiabilidad y riesgo en función del tiempo

Tema 5

Cuantificar el valor y el riesgo

Un optimizador se justifica si su valor esperado supera a su coste y si sus riesgos son aceptables y están controlados. Este tema da las herramientas para afirmarlo con números: intervalos de confianza para la mejora con diseños pareados (capítulo 2 de Montgomery [33]) y con *bootstrap* (lectura 24 del curso 18.05 del MIT [37]), tamaño de un piloto, valor actual neto (sesiones «Present Value Relations» y «Capital Budgeting» del curso 15.401 del MIT [31]), medidas de riesgo de cola (Rockafellar y Uryasev [43]), análisis de modos de fallo (IEC 60812 [20]) y el marco del Reglamento europeo de inteligencia artificial [42, 6]. Se apoya en el módulo 0.9 (temas 4 y 5), en el contraste del ahorro del módulo 26 (tema 6, ejemplo «Contraste del ahorro») y en los experimentos en línea de Kohavi, Tang y Xu [24].

5.1 Coste y beneficio

Definición 5.1 (Valor actual neto). Un proyecto con inversión inicial I y flujos netos $B_t - O_t$ (beneficios menos costes de operación) al final de cada año $t = 1, \dots, N$, con tipo de descuento anual $\rho > 0$, tiene *valor actual neto*

$$\mathrm{VAN} = -I + \sum_{t=1}^{N} \frac{B_t - O_t}{(1+\rho)^t}.$$

La *tasa interna de rentabilidad* (TIR) es el ρ que anula el VAN, y el *plazo de recuperación*, el primer año en que los flujos acumulados sin descontar cubren I.

Proposición 5.2 (Anualidad). *Si $B_t - O_t = b$ es constante,* $\mathrm{VAN} = -I + b\,a_{N,\rho}$ *con* $a_{N,\rho} = \sum_{t=1}^{N}(1+\rho)^{-t} = \dfrac{1-(1+\rho)^{-N}}{\rho}$.

Los beneficios de un optimizador se miden *respecto a una alternativa*, y la elección de la alternativa decide el resultado. La alternativa honrada no es la práctica actual, sino la mejor solución sencilla que se adoptaría sin IA. En el mantenimiento del tema 4, pasar de intervenir cada 70 días a hacerlo cada 30 (política B) no requiere sensores ni modelos de clasificación: basta un análisis de fiabilidad. El valor atribuible al optimizador con IA es la diferencia entre la política D y la B.

5.2 Intervalos de confianza para la mejora

Proposición 5.3 (Ventaja del diseño pareado). *Sean Y_A e Y_B los costes de dos políticas en el mismo periodo, con varianzas σ_A^2, σ_B^2 y correlación r. La diferencia pareada $D = Y_A - Y_B$ tiene varianza $\sigma_A^2 + \sigma_B^2 - 2r\sigma_A\sigma_B$, menor que la de la diferencia de dos medidas independientes, $\sigma_A^2 + \sigma_B^2$, si y solo si $r > 0$.*

Comparar dos políticas en los mismos días, o en el gemelo con los mismos números aleatorios (módulo 26, tema 5, §5.1), crea esa correlación positiva: la carga, el desgaste y el azar comunes se cancelan. Con n diferencias pareadas de media $\bar{d}$ y desviación típica s_d, el intervalo de confianza de nivel $1-\alpha$ para la mejora media es $\bar{d} \pm t_{n-1,1-\alpha/2}\, s_d/\sqrt{n}$ (módulo 0.9, tema 4, §4.4).

Definición 5.4 (Intervalos *bootstrap*). Sean $d_1, \dots, d_n$ las diferencias observadas y $\bar{d}$ su media. Se generan B remuestras de tamaño n con reemplazamiento, con medias $\bar{d}^*_1, \dots, \bar{d}^*_B$. El intervalo *percentil* de nivel $1-\alpha$ es $[\bar{d}^*_{(\alpha/2)}, \bar{d}^*_{(1-\alpha/2)}]$, con los cuantiles empíricos de las medias remuestreadas. El intervalo *básico* usa $\delta^* = \bar{d}^* - \bar{d}$: $[\bar{d} - \delta^*_{(1-\alpha/2)},\ \bar{d} - \delta^*_{(\alpha/2)}]$ [37].

El *bootstrap* no exige normalidad, lo que importa aquí: el coste semanal de una línea es casi siempre pequeño y, de vez en cuando, salta en 9000 € por una avería. Ambos métodos suponen observaciones independientes; si las semanas están autocorrelacionadas hay que remuestrear por bloques.

Proposición 5.5 (Tamaño de un piloto). *Para detectar una mejora media $\delta > 0$ con un contraste bilateral de nivel α y potencia $1-\beta$ con diferencias pareadas de desviación típica σ_D, hacen falta aproximadamente*

$$n \approx \Big(\frac{(z_{1-\alpha/2} + z_{1-\beta})\, \sigma_D}{\delta} \Big)^2$$

pares (módulo 0.9, tema 5, §5.4).

5.3 Caso de estudio: ¿cuánto vale el optimizador de mantenimiento?

5.4 Robustez y riesgo de cola

La media no lo es todo: a una planta le preocupa el año malo. Para una pérdida L con distribución continua, el *valor en riesgo* al nivel α es el cuantil $\mathrm{VaR}_\alpha(L) = F_L^{-1}(\alpha)$, y el *valor en riesgo condicional* es la media de la cola, $\mathrm{CVaR}_\alpha(L) = \mathbb{E}[L \mid L \geq \mathrm{VaR}_\alpha(L)]$.

Proposición 5.7 (Fórmula de Rockafellar y Uryasev). *Si L tiene densidad y esperanza finita,*

$$\mathrm{CVaR}_\alpha(L) = \min_{\tau \in \mathbb{R}} \Big\{ \tau + \frac{1}{1-\alpha} \mathbb{E}\big[(L-\tau)^+\big] \Big\},$$

y el mínimo se alcanza en $\tau = \mathrm{VaR}_\alpha(L)$.

La fórmula convierte la minimización del CVaR de una decisión en un programa lineal cuando L se representa con escenarios: basta añadir τ y una variable $u_s \geq L_s - \tau$, $u_s \geq 0$ por escenario [43]. En los 20 años simulados del caso, el coste anual de la política D tiene media 159 300 € y CVaR al 90 % (media de los dos peores años) de 186 300 €; la política B, 193 670 y 216 200 €. D es mejor también en la cola. Las *pruebas de estrés* completan el análisis: repetir la evaluación con un coste de avería un 50 % mayor, con un técnico de baja, con un sensor que se congela en su último valor o con una semana de datos perdidos.

5.5 Seguridad, supervisión y modos de fallo

Los principios del módulo 26 (tema 6) se aplican a cualquier optimizador. (i) Las *restricciones duras* (capacidad de los técnicos, inspecciones obligatorias por ley, bloqueos de seguridad de las máquinas) están en el modelo de decisión y se comprueban otra vez, de forma independiente, antes de emitir una orden. (ii) Hay una *política de respaldo* sencilla (aquí, la política por edad) a la que se vuelve si el modelo o los datos fallan. (iii) El despliegue recorre los modos asesor, supervisado y autónomo con supervisión (módulo 26, tema 6, §6.2), y en cada uno se registra quién decidió qué.

Definición 5.8 (Análisis modal de fallos y efectos). El AMFE (FMEA en inglés; IEC 60812 [20]) enumera los modos de fallo de cada componente, sus efectos y sus causas, y los prioriza. En la variante clásica se puntúan de 1 a 10 la *gravedad* S, la *ocurrencia* O y la *detectabilidad* Dt (10 = indetectable), y se ordena por el *número de prioridad de riesgo* $\mathrm{NPR} = S \cdot O \cdot Dt$; la norma admite otras formas de calcularlo y, en todo caso, los modos de gravedad máxima se tratan aunque su NPR sea bajo.

Cuadro 5.1: AMFE resumido del optimizador de mantenimiento.

Modo de fallo	Efecto	S	O	Dt	NPR	Medida
sensor congelado en su último valor	riesgo infravalorado; avería no prevista	8	3	7	168	regla de varianza nula; respaldo por edad
deriva tras cambio de producto	probabilidades mal calibradas	6	5	4	120	PSI semanal; recalibración
resolvedor sin solución a tiempo	sin plan el lunes	5	2	2	20	plan de la semana anterior; límite de tiempo
confianza excesiva del técnico	se ejecutan planes erróneos	7	4	6	168	mostrar motivos; auditoría de anulaciones
etiquetas mal registradas en el GMAO	modelo entrenado con ruido	6	4	6	144	validación de registros; revisión mensual

5.6 El Reglamento europeo de inteligencia artificial

El Reglamento (UE) 2024/1689 [42] entró en vigor el 1 de agosto de 2024 y sigue un enfoque basado en el riesgo: prohíbe algunas prácticas, impone requisitos a los sistemas de *alto riesgo*, establece obligaciones de transparencia para otros y deja el resto sin obligaciones específicas. Lo que sigue es una descripción general de su texto público; la clasificación de un sistema concreto es una cuestión jurídica.

1. *Alto riesgo* (artículo 6). Lo es un sistema que sea componente de seguridad de un producto, o el propio producto, regulado por la legislación de armonización del anexo I y sujeto a evaluación de la conformidad por terceros; y lo son los sistemas del anexo III, entre ellos los componentes de seguridad en la gestión y operación de infraestructuras críticas, como el suministro de agua, gas, calefacción o electricidad (anexo III, punto 2). Un sistema del anexo III no es de alto riesgo si no influye de forma sustancial en la decisión, por ejemplo si realiza una tarea procedimental limitada o preparatoria (artículo 6,

apartado 3), salvo que elabore perfiles de personas.

2. *Requisitos* (artículos 8 a 15): sistema de gestión de riesgos, gobernanza de los datos, documentación técnica, registro automático de eventos, transparencia hacia quien lo despliega, supervisión humana y precisión, robustez y ciberseguridad. El artículo 14 pide que las personas que supervisan puedan entender las capacidades y los límites del sistema y detectar anomalías, sean conscientes del sesgo de automatización, interpreten correctamente sus salidas, puedan decidir no usarlo o anular sus resultados, e intervenir o detenerlo de forma segura.
3. *Calendario.* El reglamento se aplica con carácter general desde el 2 de agosto de 2026. La modificación conocida como «ómnibus digital», publicada en el Diario Oficial el 24 de julio de 2026 y en vigor desde el 27 de julio, aplaza las obligaciones de alto riesgo: al 2 de diciembre de 2027 para los sistemas del anexo III y al 2 de agosto de 2028 para los del anexo I; según la información publicada, la maquinaria pasa a un régimen sectorial propio [6, 42].

Aplicado al caso: un optimizador del calendario de mantenimiento de una línea de envasado, cuyas órdenes revisa un técnico y que no actúa sobre las funciones de seguridad de las máquinas, no parece encajar en ninguna categoría de alto riesgo. El mismo optimizador aplicado a las bombas de una red de abastecimiento de agua, si actuara como componente de seguridad de su operación, sí podría serlo. En ambos casos, la arquitectura de los temas 4 y 5 (registro de decisiones, supervisión humana con capacidad de anular, política de respaldo, monitorización, documentación) cumple de forma natural buena parte de los requisitos, y es buena práctica adoptarla aunque no sea obligatoria. Si se usan datos de personas (por ejemplo, qué técnico anuló qué orden), se aplica además la normativa de protección de datos.

Cuadro 5.2: Tecnologías del tema 5.

Herramienta	Para qué sirve	Licencia o tipo	Cuándo elegirla
SciPy (`stats`)	contrastes t, intervalos y *bootstrap* (función `bootstrap`)	abierta (BSD-3)	análisis de pilotos y de simulaciones
statsmodels	modelos con errores autocorrelacionados, potencia y tamaño muestral	abierta (BSD-3)	series semanales, diseños con covariables
numpy-financial	VAN, TIR y anualidades	abierta (BSD-3)	valoración económica reproducible
MLflow [10]	registrar cada evaluación con su modelo, datos y resultado	abierta (Apache 2.0)	trazabilidad exigida por la documentación técnica
Plantillas de AMFE (hoja de cálculo)	análisis de modos de fallo	—	revisión de riesgos con operación, mantenimiento y seguridad

Tema 6

Guía para construir un optimizador con IA en cualquier ámbito

Este tema reúne la fase VI en un procedimiento. No es un algoritmo, sino una disciplina de ingeniería: el orden en que conviene tomar las decisiones y los documentos que deben quedar escritos. Sigue la estructura del curso 16.842 «Fundamentals of Systems Engineering» del MIT [30], que recorre el modelo en V (partes interesadas y concepto de operación, requisitos con márgenes, arquitectura, exploración del espacio de alternativas, revisión preliminar del diseño), las clases 1 y 2 del curso CS329S de Stanford [46] y el capítulo 2 de Huyen [18] para el planteamiento de sistemas con aprendizaje automático, y la lista de pruebas de Breck *et al.* [5]. Cada paso remite al módulo donde se estudió su contenido.

6.1 Los doce pasos

1. **La decisión y sus responsables.** ¿Qué se decide, quién lo decide hoy, con qué frecuencia y con qué horizonte? ¿Quién usará el sistema, quién responde de sus errores y quién puede pararlo? Entregable: concepto de operación de una página.
2. **Objetivo, indicadores y restricciones.** Un objetivo escalar, los indicadores que se vigilan sin optimizarlos, las restricciones duras (seguridad, ley, física) y las blandas, cada una con su unidad y su fuente (módulo 24, tema 1). Entregable: requisitos numéricos con márgenes.
3. **Línea base y techo de valor.** Coste de la práctica actual, de la mejor alternativa sencilla sin IA y de una política con información perfecta (proposición 6.1). Si el techo no compensa el coste del proyecto, se para aquí.
4. **Datos.** Inventario de fuentes, frecuencias, calidad, retrasos, disponibilidad en el instante de decidir, propiedad y protección; versionado desde el primer día (tema 4).
5. **Formulación y familia de métodos.** El mapa del módulo 24 (tema 1, §1.3): programación matemática, optimización bayesiana para experimentos caros (módulo 25), control y RL para decisiones secuenciales (módulo 26), optimización geométrica (módulo 27), búsqueda de programas cuando lo que se busca es una regla (tema 1).
6. **Modelos predictivos o sustitutos.** Qué se predice, con qué horizonte y con qué métrica de decisión; validación temporal y selección con presupuesto (tema 3).
7. **Resolvedor, salvaguardas y respaldo.** Tiempo máximo de cálculo, comprobación independiente de las restricciones duras, política de respaldo.
8. **Evaluación fuera de línea.** Sobre históricos (sin fugas), en un gemelo con números aleatorios comunes y, si hay registros con propensiones, con estimadores contrafactuales (tema 4).

9. **Piloto.** Modo sombra y después experimento con tamaño calculado (proposición 5.5) y criterio de aceptación escrito antes de empezar.
10. **Despliegue.** API con contrato, interfaz que explica y permite anular, registro de decisiones, versiones y vuelta atrás.
11. **Monitorización y reentrenamiento.** Entradas, predicciones, decisiones, anulaciones e indicadores; disparadores y periodo de reentrenamiento (proposición 4.8).
12. **Valor, riesgo, normativa y ciclo de vida.** Intervalos para la mejora, VAN, AMFE, encaje regulatorio, documentación y criterios de retirada (tema 5).

Proposición 6.1 (Techo de valor). *Sea $f(x,\theta)$ el coste de la decisión $x \in \mathcal{X}$ cuando el estado incierto es θ, y sea $\hat{x}$ cualquier regla de decisión que solo use información disponible antes de decidir. Entonces*

$$\mathbb{E}\big[f(\hat{x},\theta)\big] \;\geq\; \mathbb{E}\Big[\min_{x\in\mathcal{X}} f(x,\theta)\Big].$$

En consecuencia, ninguna mejora de la predicción puede reducir el coste esperado de la mejor regla sin información por debajo del coste con información perfecta, y la diferencia entre ambos (el valor de la información perfecta del módulo 24, tema 6, definición 6.6) acota lo que puede ganarse prediciendo mejor.

En el caso del mantenimiento, una política que conoce el futuro e interviene cada máquina en la última ventana antes de que fuera a fallar cuesta de media 76 170 €/año en el gemelo (sin límite de técnicos), frente a 193 670 de la política por edad y 159 300 del optimizador. Este solo captura un 29 % del margen que deja la alternativa sencilla frente a la información perfecta: hay espacio para mejores sensores o mejores modelos, y el paso 3 permite estimar cuánto valdrían antes de pagarlos.

6.2 Plantilla de documento de diseño

El cuadro 6.1 es la plantilla que se rellena antes de escribir código y se actualiza en cada revisión. Su objetivo no es la burocracia, sino forzar que las decisiones difíciles (qué es éxito, qué no se puede violar, contra qué se compara) se tomen pronto y por escrito, como en la revisión preliminar de diseño de 16.842 [30].

6.3 Mini-caso A: secuenciación de formatos en una fábrica de bebidas

6.4 Mini-caso B: presupuesto de geometría en un videojuego

6.5 Mini-caso C: asignación de trabajos en un centro de datos

Cuadro 6.1: Plantilla del documento de diseño de un optimizador con IA.

Sección	Preguntas que debe responder
1. Contexto y decisión	¿Qué se decide, cada cuánto, con qué horizonte? ¿Qué pasa hoy?
2. Partes interesadas	¿Quién decide, quién ejecuta, quién responde, quién puede parar el sistema?
3. Objetivo y requisitos	Objetivo, indicadores, restricciones duras y blandas con unidades; requisitos de latencia, disponibilidad y precisión con márgenes.
4. Línea base y techo	Coste actual, mejor alternativa sencilla, coste con información perfecta; valor máximo alcanzable.
5. Datos	Fuentes, calidad, retrasos, disponibilidad en el instante de decidir, versionado, protección.
6. Formulación y método	Modelo matemático; familia elegida y alternativas descartadas con su motivo.
7. Modelos y validación	Qué se predice; división temporal; métricas de predicción y de decisión; selección.
8. Salvaguardas y respaldo	Comprobaciones independientes; política de respaldo; tiempo máximo de cálculo.
9. Evaluación fuera de línea	Históricos, gemelo, contrafactual; resultados con intervalos.
10. Piloto	Diseño, tamaño, duración, criterio de aceptación fijado de antemano.
11. Despliegue y supervisión	API, interfaz, modos de operación, registro, vuelta atrás.
12. Monitorización	Señales, umbrales, acciones, reentrenamiento.
13. Valor y riesgo	VAN con incertidumbre, AMFE, pruebas de estrés, encaje normativo.
14. Ciclo de vida	Responsable del mantenimiento, revisiones periódicas, criterios de retirada.

Cuadro 6.2: Tecnologías del tema 6: pila de referencia de un optimizador con IA.

Capa	Herramientas habituales	Licencia o tipo	Cuándo elegirla
Datos y versionado	DVC [7], Great Expectations	abiertas (Apache 2.0)	desde el primer prototipo
Modelos	scikit-learn, PyTorch; Optuna [1] para el ajuste	abiertas (BSD-3; BSD-3; MIT)	según el tipo de datos (tema 3)
Decisión	SciPy/HiGHS, OR-Tools, CVXPY (módulo 24); BoTorch (módulo 25)	abiertas	según la estructura del problema
Búsqueda con LLM	OpenEvolve [35], DSPy [22]	abiertas (Apache 2.0; MIT)	reglas o cadenas de llamadas con evaluador fiable
Servicio	FastAPI [9], contenedores, Kubernetes [25]	abiertas (MIT; Apache 2.0)	integración con otros sistemas
Seguimiento	MLflow [10], Evidently [8]	abiertas (Apache 2.0)	trazabilidad y deriva
Escala	Ray [40]	abierta (Apache 2.0)	simulaciones y ajustes en paralelo

Bibliografía

[1] T. Akiba, S. Sano, T. Yanase, T. Ohta y M. Koyama. Optuna: A next-generation hyperparameter optimization framework. En *Proceedings of the 25th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2019. Documentación oficial; licencia MIT. https://optuna.org/

[2] R. E. Barlow y F. Proschan. *Mathematical Theory of Reliability*. SIAM, colección Classics in Applied Mathematics, 1996 (original: Wiley, 1965). Cap. 4, «Optimum Maintenance Policies».

[3] J. Bergstra e Y. Bengio. Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13:281–305, 2012.

[4] D. Bertsimas y J. N. Tsitsiklis. *Introduction to Linear Optimization*. Athena Scientific, 1997. Cap. 7, «Network flow problems».

[5] E. Breck, S. Cai, E. Nielsen, M. Salib y D. Sculley. The ML test score: A rubric for ML production readiness and technical debt reduction. En *IEEE International Conference on Big Data*, 2017.

[6] CMS. The EU's Digital Omnibus: streamlined AI Act compliance rules are now in force. Nota informativa, 2026 (publicación en el Diario Oficial el 24 de julio de 2026; entrada en vigor el 27 de julio de 2026). https://cms.law/en/gbr/publication/the-eu-s-digital-omnibus-streamlined-ai-act-compliance-rules-are-now-in-force

[7] Desarrolladores de DVC. DVC: Data Version Control. Documentación oficial; licencia Apache 2.0. https://dvc.org/doc

[8] Desarrolladores de Evidently. Evidently: evaluación y monitorización de modelos de aprendizaje automático. Documentación oficial; licencia Apache 2.0. https://docs.evidentlyai.com/

[9] Desarrolladores de FastAPI. FastAPI. Documentación oficial; licencia MIT. https://fastapi.tiangolo.com/

[10] Desarrolladores de MLflow. MLflow: plataforma para el ciclo de vida del aprendizaje automático. Documentación oficial; licencia Apache 2.0. https://mlflow.org/docs/latest/

[11] M. Dudík, J. Langford y L. Li. Doubly robust policy evaluation and learning. En *Proceedings of the 28th International Conference on Machine Learning (ICML)*, 2011. https://arxiv.org/abs/1103.4601

[12] T. Elsken, J. H. Metzen y F. Hutter. Neural architecture search. Cap. 3 de [19], 2019.

[13] R. Evans y J. Gao. DeepMind AI reduces Google data centre cooling bill by 40 %. Blog de Google DeepMind, 20 de julio de 2016. https://deepmind.google/discover/blog/deepmind-ai-reduces-google-data-centre-cooling-bill-by-40/

[14] Google DeepMind (equipo de AlphaEvolve). AlphaEvolve: A Gemini-powered coding agent for designing advanced algorithms. Blog de Google DeepMind, 14 de mayo de 2025. https://deepmind.google/discover/blog/alphaevolve-a-gemini-powered-coding-agent-for-designing-advanced-algorithms/

[15] A. Fawzi y B. Romera-Paredes. FunSearch: Making new discoveries in mathematical sciences using large language models. Blog de Google DeepMind, 14 de diciembre de 2023 (con una actualización de 2024). https://deepmind.google/discover/blog/funsearch-making-new-discoveries-in-mathematical-sciences-using-large-language-models/

[16] Google DeepMind. FunSearch: repositorio oficial (implementación del algoritmo evolutivo, cuaderno `bin_packing` con los conjuntos OR3 y Weibull 5k y las heurísticas descubiertas). Licencia Apache 2.0. https://github.com/google-deepmind/funsearch

[17] J. Herrmann y G. Pallez. An in-depth study of LLM contributions to the bin packing problem. Prepublicación, 2025. https://arxiv.org/abs/2510.27353

[18] C. Huyen. *Designing Machine Learning Systems*. O'Reilly, 2022. Texto asociado al curso CS329S de Stanford.

[19] F. Hutter, L. Kotthoff y J. Vanschoren (eds.). *Automated Machine Learning: Methods, Systems, Challenges*. Springer, 2019. Edición de acceso abierto. Cap. 1 «Hyperparameter Optimization» (M. Feurer y F. Hutter) y cap. 3 «Neural Architecture Search». https://www.automl.org/book/

[20] IEC. IEC 60812:2018. *Failure modes and effects analysis (FMEA and FMECA)*, 3.ª ed. International Electrotechnical Commission, 2018. Se usa solo el resumen público. https://webstore.iec.ch/en/publication/26359

[21] K. Jamieson y A. Talwalkar. Non-stochastic best arm identification and hyperparameter optimization. En *Proceedings of the 19th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2016. https://arxiv.org/abs/1502.07943

[22] O. Khattab, A. Singhvi, P. Maheshwari, Z. Zhang, K. Santhanam, S. Vardhamanan, S. Haq, A. Sharma, T. T. Joshi, H. Moazam, H. Miller, M. Zaharia y C. Potts. DSPy: Compiling declarative language model calls into self-improving pipelines. Prepublicación, 2023. Biblioteca con licencia MIT. https://arxiv.org/abs/2310.03714

[23] M. J. Kochenderfer y T. A. Wheeler. *Algorithms for Optimization*. MIT Press, 2019. Cap. 9 «Population Methods» y cap. 20 «Expression Optimization». https://algorithmsbook.com/optimization/

[24] R. Kohavi, D. Tang y Y. Xu. *Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing*. Cambridge University Press, 2020.

[25] The Kubernetes Authors. Kubernetes. Documentación oficial; licencia Apache 2.0. https://kubernetes.io/docs/

[26] L. Li y A. Talwalkar. Random search and reproducibility for neural architecture search. En *Proceedings of the 35th Conference on Uncertainty in Artificial Intelligence (UAI)*, 2019. https://arxiv.org/abs/1902.07638

[27] L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh y A. Talwalkar. Hyperband: A novel bandit-based approach to hyperparameter optimization. *Journal of Machine Learning Research*, 18(185):1–52, 2018.

[28] H. Liu, K. Simonyan e Y. Yang. DARTS: Differentiable architecture search. En *International Conference on Learning Representations (ICLR)*, 2019. https://arxiv.org/abs/1806.09055

[29] MIT OpenCourseWare. 1.010: Uncertainty in Engineering. Massachusetts Institute of Technology, otoño de 2008. *Example Application* 8, «Old better than new, new better than old…» (función de riesgo), problema 8.1. https://ocw.mit.edu/courses/1-010-uncertainty-in-engineering-fall-2008/resources/app_08/

[30] O. de Weck. 16.842: Fundamentals of Systems Engineering. Massachusetts Institute of Technology, MIT OpenCourseWare, otoño de 2015. Programa (modelo en V) y tareas A1 (partes interesadas, concepto de operación), A2 (requisitos y asignación de márgenes) y A4 (exploración de alternativas y selección del concepto). https://ocw.mit.edu/courses/16-842-fundamentals-of-systems-engineering-fall-2015/

[31] A. Lo. 15.401: Finance Theory I. Massachusetts Institute of Technology, MIT OpenCourseWare, otoño de 2008. Sesiones «Present Value Relations» y «Capital Budgeting». https://ocw.mit.edu/courses/15-401-finance-theory-i-fall-2008/

[32] C. Leiserson y J. Shun. 6.172: Performance Engineering of Software Systems. Massachusetts Institute of Technology, MIT OpenCourseWare, otoño de 2018. Clase 10, «Measurement and Timing» (transcripción). https://ocw.mit.edu/courses/6-172-performance-engineering-of-software-systems-fall-2018/

[33] D. C. Montgomery. *Design and Analysis of Experiments*, 9.ª ed. Wiley, 2017. Cap. 2, «Simple Comparative Experiments».

[34] A. Novikov, N. Vũ, M. Eisenberger, E. Dupont, P.-S. Huang, A. Z. Wagner *et al.* AlphaEvolve: A coding agent for scientific and algorithmic discovery. Prepublicación, Google DeepMind, 2025. https://arxiv.org/abs/2506.13131

[35] Desarrolladores de OpenEvolve. OpenEvolve: implementación abierta inspirada en AlphaEvolve. Repositorio oficial; licencia Apache 2.0. https://github.com/algorithmicsuperintelligence/openevolve

[36] K. Opsahl-Ong, M. J. Ryan, J. Purtell, D. Broman, C. Potts, M. Zaharia y O. Khattab. Optimizing instructions and demonstrations for multi-stage language model programs. En *Proceedings of EMNLP*, 2024. https://arxiv.org/abs/2406.11695

[37] J. Orloff y J. Bloom. 18.05: Introduction to Probability and Statistics. Massachusetts Institute of Technology, MIT OpenCourseWare, primavera de 2022. Lectura 24, «Bootstrap confidence intervals». https://ocw.mit.edu/courses/18-05-introduction-to-probability-and-statistics-spring-2022/

[38] H. Pham, M. Y. Guan, B. Zoph, Q. V. Le y J. Dean. Efficient neural architecture search via parameter sharing. En *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018. https://arxiv.org/abs/1802.03268

[39] C. Potts. CS224U: Natural Language Understanding. Stanford University, primavera de 2023. Tarea «Few-shot OpenQA with DSPy». https://web.stanford.edu/class/cs224u/

[40] P. Moritz, R. Nishihara, S. Wang *et al.* Ray: A distributed framework for emerging AI applications. En *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2018. Documentación de Ray Tune y Ray Serve; licencia Apache 2.0. https://docs.ray.io/

[41] E. Real, A. Aggarwal, Y. Huang y Q. V. Le. Regularized evolution for image classifier architecture search. En *Proceedings of the AAAI Conference on Artificial Intelligence*, 2019. https://arxiv.org/abs/1802.01548

[42] Parlamento Europeo y Consejo de la Unión Europea. Reglamento (UE) 2024/1689, de 13 de junio de 2024, por el que se establecen normas armonizadas en materia de inteligencia artificial (Reglamento de Inteligencia Artificial). *Diario Oficial de la Unión Europea*, 12 de julio de 2024. Texto consultado, con sus modificaciones, en https://artificialintelligenceact.eu/ (artículos 6, 14 y 113; anexos I y III).

[43] R. T. Rockafellar y S. Uryasev. Optimization of conditional value-at-risk. *Journal of Risk*, 2(3):21–41, 2000.

[44] B. Romera-Paredes, M. Barekatain, A. Novikov, M. Balog, M. P. Kumar, E. Dupont, F. J. R. Ruiz, J. S. Ellenberg, P. Wang, O. Fawzi, P. Kohli y A. Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024.

[45] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo y D. Dennison. Hidden technical debt in machine learning systems. En *Advances in Neural Information Processing Systems 28 (NIPS)*, 2015.

[46] C. Huyen y equipo docente de CS329S. CS329S: Machine Learning Systems Design. Stanford University, invierno de 2022. Clases 1 «Understanding machine learning production», 2 «ML and Data Systems Fundamentals», 6 «Offline evaluation», 8 «Deployment» y 10 «Diagnosis of ML system failures, data distribution shifts & monitoring», con notas de clase. https://stanford-cs329s.github.io/syllabus.html

[47] J. E. Smith y R. L. Winkler. The optimizer's curse: Skepticism and postdecision surprise in decision analysis. *Management Science*, 52(3):311–322, 2006.

[48] C. Thornton, F. Hutter, H. H. Hoos y K. Leyton-Brown. Auto-WEKA: Combined selection and hyperparameter optimization of classification algorithms. En *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2013.

[49] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery y D. Zhou. Self-consistency improves chain of thought reasoning in language models. En *International Conference on Learning Representations (ICLR)*, 2023. https://arxiv.org/abs/2203.11171

[50] D. P. Williamson y D. B. Shmoys. *The Design of Approximation Algorithms*. Cambridge University Press, 2011. Edición electrónica libre. §3.3, «The bin-packing problem». https://www.designofapproxalgs.com/

[51] C. Yang, X. Wang, Y. Lu, H. Liu, Q. V. Le, D. Zhou y X. Chen. Large language models as optimizers. Prepublicación, 2023 (versión 3, 2024). https://arxiv.org/abs/2309.03409

[52] A. Zhang, Z. C. Lipton, M. Li y A. J. Smola. *Dive into Deep Learning*, versión 1.0.3. Cambridge University Press, 2023. §19.1 «What Is Hyperparameter Optimization?» y §19.4 «Multi-Fidelity Hyperparameter Optimization». https://d2l.ai/

[53] B. Zoph y Q. V. Le. Neural architecture search with reinforcement learning. En *International Conference on Learning Representations (ICLR)*, 2017. https://arxiv.org/abs/1611.01578